Deliverables

1. A complete software package
2. A single script that runs all experiments in “Table: Summary of results”
3. A single script that runs your algorithm of choice” on the Rosenbrock function, with your optimal choice of parameters
4. Poster (due 4/16)
5. Report (due 4/21)

To do

1. Write script for the 10 algorithms
2. Ensure that the algorithms run for the test problems provided on canvas
3. Write script for the 10 functions and their gradients and hessians
4. Implement each algorithm for each problem (Progress: Put a check in the box when the algorithm is run on the functions)

	GD	GDW	Newton	NewtonW	TRNewtonCG	TRSR1CG	BFGS	BFGSW	DFP	DFPW
P1	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P2	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P3	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P4	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P5	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P6	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P7	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P8	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P9	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P10	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P11	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
P12	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Report

Algorithm Summary

Summarize each algorithm in a few sentences. Should include:

- Whether it is a trust region or line search algorithm
- How the steps are computed
- How non convexity is handled

#TODO: add text here

Default options for our code

Provide a table of default options for your code, which may or may not vary between algorithms.
#TODO: add table of default options

Results

Compare the performance of the algorithms on the given problems. #TODO: fill in tables
Table: Summary of Results

	1. GradientDescent, with backtracking line search				2. GradientDescentW, with Wolfe line search			
Problem	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds
1								
2								
3								
4								
5								
6								
7								

8								
9								
10								

	3. Newton, (modified Newton) with backtracking line search				4. NewtonW, (modified Newton) with Wolfe line search			
Problem	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								

	5. TRNewtonCG, trust region Newton with CG subproblem solver				6. TRSR1CG, SR1 quasi-Newton with CG subproblem solver			
Problem	Iterations	Time	Number of Function/Gradient/Hes	CPU seconds	Iterations	Time	Number of Function/Gradient/Hes	CPU seconds

			sian				sian	
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								

	7. BFGS, BFGS quasi-Newton with backtracking line search				8. BFGSW, BFGS quasi-Newton with Wolfe line search			
Problem	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds
1								
2								
3								
4								
5								
6								
7								
8								

9								
10								

	9. DFP, DFP quasi-Newton with backtracking line search				10. DFPW, DFP quasi-Newton with Wolfe line search			
Problem	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds	Iterations	Time	Number of Function/Gradient/Hessian	CPU seconds
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								

I. Compare the algorithms using any of the techniques studied in class (Function vs. iteration plots, performance profiles, etc.) #TODO: answer this question
II. Comment on the results in your output table: #TODO: answer these questions
    A. Was any algorithm consistently the best?
    B. Why did some algorithms have trouble with certain problems?
III. If you had to choose one algorithm (that balances convergence, speed, etc.), which algorithm would be your “algorithm of choice?” Why? #TODO: answer this question

Our “big question”

IV. Investigate one big question. #TODO: answer this question

Objective and Scope:

We decided to investigate the bigger question of *what are good choices of the line search parameters*. To accomplish this, we will systematically test different Armijo line search parameters ($c_{1,\,ls}$ and $c_{2,\,ls}$) and backtracking line search parameters ($\bar{\alpha}$, τ, c_1). We will evaluate these parameters across the different given problems, using the different methods to find the moving directions. We will utilize a broad range of parameter values that includes both common chosen values and extreme bounds to observe their respective convergence speed failure rates across different algorithms and problems.

Summary

I. Would you declare any of your algorithms the winner? Consider: #TODO: answer this question
   A. The algorithm's performance
   B. How easy it was to code
   C. How many parameters you need to tune before it worked well
   D. What method would you recommend to an expert coder?
   E. What method would you recommend to a user who is not an expert in coding or in nonlinear optimization? How does that depend on whether or not the user is able to code second derivatives for their problem?